Документация языка sigil+BTB

Протокол общения между пользователем и ИИ-ассистентом

Документ — технический мануал

Аудитория: начинающий программист

Версия документа: v1.5 (& sigil + code booster v1.1 + first booster cycle + v49 milestone)

Дата: 31 октября 2026

Оглавление

Оглавление....2

1. Введение — зачем нужен этот язык....4

2. Основные идеи простыми словами....4

2.1. Аналогия с цветными стикерами....4

2.2. Аналогия с кулинарной привычкой (и процессором)....5

3. Сигилы — пять символов....6

4. Команды действий (!) — глаголы....6

4.1. Обязательность сигила и глагола....7

4.2. Глаголы для & (предложения/обсуждение методов, Task 62)....8

5. Теги состояния (@) — автотокены....9

6. meta.auto — файл автоподстановки....9

7. BTB — предзагрузка по памяти....10

7.1. Как это работает....10

7.2. Почему это безопасно....11

7.3. Как обновляется BTB — файловый цикл (read-modify-write)....11

7.5. #stats — метрики BTB....12

7.4. Честный предел....13

7.6. BTB self-tuning: уточнение сигнатур через #proto....13

8. Полный синтаксис запроса !feat (и !remove)....13

8.1. after: как исполняемый предикат (DSL)....14

9. Примеры использования....15

9.1. Простой вопрос....15

9.2. Новая функция (минимальная форма)....15

9.3. Новая функция (детальная форма)....15

9.4. Удаление функции (новое в v0.2)....16

9.5. Диагностика сбоя....16

9.6. Откат к последней рабочей версии....16

9.7. Заморозка и новое правило....16

10. Правила взаимодействия....17

10.1. Kill-switch — ОТКЛОНЁН (Tasks 64 + reqs_killswitch.md)....18

11. Словарь терминов....18

12. Краткая шпаргалка....21

Сигилы....22

Глаголы для ! (обязательны внутри !)....22

Глаголы для & (Task 62, v1.5)....22

Мета-команды #....22

Автотокены (вместо чисел и хешей)....22

Три правила....22

Минимальный запрос на новую функцию....23

13. v2 reforms summary (Tasks 49-56)....23

13.1. v1.5 расширение (Tasks 57-66)....24

14. Code booster v1.1 — GLSL style statistics....27

14.1. Booster v1.0 → v1.1 — 6 расширений (Task 64)....27

14.2. 5-layer validation pipeline....28

14.3. query_booster.py — CLI interface....28

14.4. GLSL validator pipeline (Task 59)....29

14.5. First end-to-end cycle (Task 65 — v49 upper-cyan-fix)....29

14.6. BTB entry для !fix *shader* (first measured)....30

15. v1.5 changelog — что нового (Tasks 57-66)....31

Оглавление обновится после правого клика → «Обновить поле».

1. Введение — зачем нужен этот язык

Эта документация описывает небольшой язык, который мы придумали, чтобы работать вместе быстрее и точнее. Он называется sigil+BTB. Не пугайтесь названия — внутри нет ничего сложного. По сути это набор коротких пометок, которые вы ставите в начале сообщения, чтобы я (ассистент) сразу понимал, чего вы хотите, без угадываний и лишних вопросов.

Зачем это понадобилось. Мы долго работали над проектом — графическое демо на C++ и OpenGL. В процессе мы столкнулись с тремя неприятными вещами, которые съедали время:

Во-первых, неоднозначность. Вы писали «не работает» — а я не понимал, что именно: не компилируется, запускается и падает, или показывает чёрный экран. Каждый такой запрос порождал длинный обмен сообщениями, пока мы не находили причину.

Во-вторых, потеря работы. Моё рабочее окружение (песочница) однажды откатилось к старому состоянию, и все промежуточные результаты исчезли. Пришлось восстанавливать из вашей локальной копии.

В-третьих, медленный старт каждого шага. Прежде чем что-то сделать, мне приходилось заново читать файлы, проверять версии, сверять контрольные суммы — по одному действию за раз.

Язык sigil+BTB решает все три проблемы сразу. Он состоит из двух частей:

sigil (символ-метка) — это четыре значка в начале строки, которые говорят мне тип сообщения: вопрос, действие, факт или правило. **BTB** (Branch Target Buffer, «буфер адресов переходов» — термин из устройства процессора) — это моя внутренняя привычка заранее подгружать нужные файлы, опираясь на то, как выглядели ваши прошлые запросы.

Дальше в документе всё объясняется простыми словами, с аналогиями из повседневной жизни и диаграммами. Если встретите незнакомый термин — он тут же расшифровывается в скобках, а полный словарь собран в разделе 12.

2. Основные идеи простыми словами

2.1. Аналогия с цветными стикерами

Представьте, что мы обмениваемся записками. Обычно записка — это просто предложение, которое надо прочитать и понять, чего хочет автор. Это медленно и иногда неточно. Теперь представьте, что на каждой записке наклеен цветной стикер:

синий — «я задаю вопрос, просто поговори со мной»; **красный** — «сделай действие, вот задача»; **зелёный** — «держи сведения, это не просьба, а факты»; **серый** — «измени правило или подтверди».

Стикер необязателен — можно писать и обычной речью, я пойму по смыслу. Но когда стикер есть, я вижу тип сообщения мгновенно и не гадаю. В нашем языке роль стикеров играют четыре символа (сигила): ?, !, @, #. Какой выбрать — подскажет диаграмма ниже.

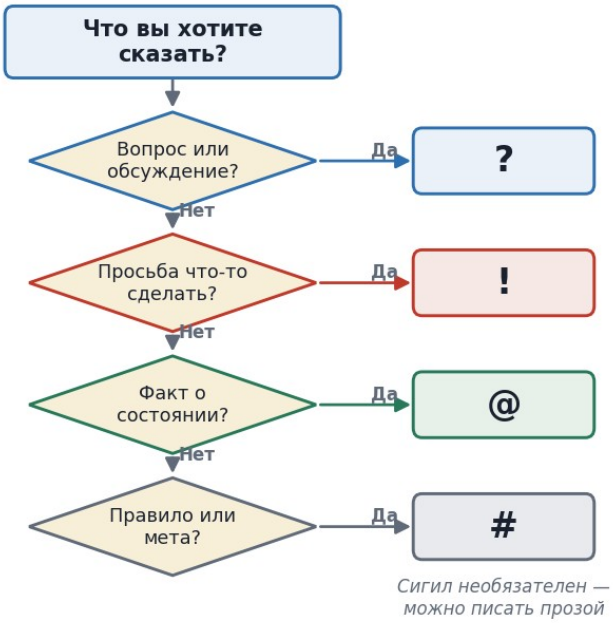


Диаграмма 1. Выбор сигила — простое дерево решений.

2.2. Аналогия с кулинарной привычкой (и процессором)

Вторая идея чуть хитрее, но тоже из жизни. Когда вы часто готовите одно и то же блюдо, вы замечаете закономерность: «если делаю пасту — понадобится кипяток, соль и кастрюля». В следующий раз, только сказав себе «паста», вы уже ставите воду, не дожидаясь, пока перечитаете рецепт. Вы запомнили шаблон и готовитесь заранее.

Процессор вашего компьютера делает то же самое. Внутри него есть специальный буфер (он и называется BTB — Branch Target Buffer), который запоминает: «после такой вот точки выбора программа прошлый раз пошла вот туда». И в следующий раз процессор заранее подтягивает нужные инструкции (команды), не дожидаясь, пока выбор разрешится. Если угадал — сэкономил время. Если промахнулся — выбрасывает подтянутое, вреда нет.

Я поступаю так же. У меня есть маленькая табличка-память с шаблонами ваших сообщений: «когда вы просите починить шейдер — мне понадобятся вот эти файлы». Когда приходит ваше сообщение, я сразу начинаю читать эти файлы несколько штук одновременно (параллельно), пока ещё разбираю ваши точные слова. К моменту, как я понял задачу, файлы уже загружены. Угадал — сэкономил. Промахнулся — выкидываю прочитанное (это бесплатно), делаю аккуратно. Подробно — в разделе 7.

3. Сигилы — пять символов

Сигил — это символ в самом начале строки. Он обозначает тип строки. Вот все пять:

Сигил	Назначение	Пример	Связь с правилами
?	Вопрос или обсуждение. Не требует файла-отчёта (trace.log).	?почему вариадик-указатель ломает float ABI?	Приостанавливает правило «нет trace.log → отмена».
!	Действие — просьба что-то сделать: починить, добавить, удалить, откатить, диагностировать.	!feat nebula background in main fragment shader	Если правка кода и trace.log не прикреплён — блок «Где trace.log? Атдай!».
@	Факт, сведения о состоянии. Не просьба, а контекст, который я записываю и сверяю.	@v=maxTask @md5=latestZip @status=working	Используется для сверки: если мой архив не совпадает с вашим — песочница откатилась.
#	Мета: новое правило, подтверждение, заморозка контрольной точки, изменение протокола.	#rule 15: … или #milestone v45-working	Записывается в журнал памяти проекта, переживает перезапуск песочницы.
&	Предложение/ обсуждение методов оптимизации работы, предложений по улучшению (Task 62). trace.log НЕ нужен.	&plan что занимает больше времени при написании GLSL? или &suggest kill-switch design	BTB delta=[] (base-only, как ? и #stats-retired). Не требует сборки/лога.

Таблица 1. Пять сигилов и их назначение (v1.5: добавлен & из Task 62).

Сигил можно не ставить — сообщение, написанное обычной прозой, я понимаю по смыслу. Сигил нужен тогда, когда вы хотите однозначности и скорости.

4. Команды действий (!) — глаголы

Сигил «!» говорит «сделай действие». Уточнить, какое именно, помогает глагол, который идёт сразу за восклицательным знаком. Вот полный список глаголов:

Глагол	Что делает	trace.log?	Пример
!fix	Хирургическая правка конкретного места.	Да	!fix main.cpp: убрать варнинг C4996

Глагол	Что делает	trace.log?	Пример
!feat	Новая функция или возможность.	Да	!feat nebula background in main fragment shader
!remove	Хирургическое удаление одной фичи (код + проводка), остальное сохраняется. Новое в v0.2.	Да	!remove nebula background from main fragment shader
!revert	Откат всей версии целиком (по имени или отпечатку).	Нет	!revert lastgood
!diag	Диагностика сбоя по симптому.	Да (обязательно)	!diag чёрный экран после «Window shown»
!snapshot	Заморозить контрольную точку — копию, которую не перезаписываю.	Нет	!snapshot v45-nebula-working
!verify	Сверить контрольную сумму готовой работы.	Нет	!verify 6a415cb0…
!clean	Убрать старые промежуточные файлы.	Нет	!clean

Таблица 2. Восемь глаголов для сигила «!».

Замечание: пользовательские макросы (!macro — цепочка глаголов одним словом) рассматривались (Task 53), но ОТКЛОНЕНЫ. В протоколе остаётся 8 глаголов выше. Принцип: не расширять про запас; макросы добавим, если реальная потребность не уложится в существующие глаголы. Multi-turn workflows (!workflow) — тоже отложены.

Глаголы, помеченные «trace.log? Да», требуют, чтобы вы прикрепили файл-отчёт сборки и запуска. Если не прикрепили — срабатывает блок «Где trace.log? Атдай!» и запрос отменяется. Это сделано специально: вслепую, без отчёта, я могу только портить, угадывая наугад.

4.1. Обязательность сигила и глагола

Мы договорились о двух простых правилах, которые убирают неоднозначность:

1. Сигил обязателен в каждом формальном запросе. Если вы пишете прозой (без сигила) — я трактую по смыслу, но без гарантии однозначности, и с риском срабатывания правила 13, если запрос похож на правку кода без trace.log.

2. Внутри «!» глагол обязателен. Голое «!сделай мне красиво» недопустимо — выберите глагол (!feat, !fix, !remove …). Если неясно какой — я спрошу. Для остальных сигил (?, @, #) глагол не нужен: сам сигил уже задаёт тип действия.

Тип строки	Сигил	Глагол	Что обязательно
Вопрос/обсуждение	?	—	Только сигил ?
Действие	!	fix/feat/remove/ revert/diag/snapshot/ verify/clean	Сигил ! И глагол
Факт/состояние	@	—	Только сигил @
Мета	#	—	Только сигил #
Проза (без сигила)	—	—	Допустимо, но риск rule 13

Таблица 3. Что обязательно в каждом типе запроса.

4.2. Глаголы для & (предложения/обсуждение методов, Task 62)

Сигил «&» обозначает предложение или обсуждение методов оптимизации работы. Внутри «&» глагол обязателен (как и для «!»). Вот три глагола:

Глагол	Что делает	trace.log?	Пример
&suggest	Предложить улучшение, новую идею, расширение протокола.	Нет	&suggest kill-switch для отказа от артефактов протокола
&review	Обзор текущего состояния, ретроспектива проделанной работы.	Нет	&review booster v1.1 — что работает, что нет
&plan	Обсуждение методов, планирование подхода к задаче.	Нет	&plan detailed — как модифицировать booster, чтобы использовать целиком

Таблица 2b. Три глагола для сигила «&» (добавлены в v1.5, Task 62).

Все три глагола & не требуют trace.log — это мета-обсуждение, не действия над кодом. BTB delta=[] (base-only) — для & нет signature-specific prefetch, как и для ?.

Принцип: & не порождает код или изменения файла — только обсуждение. Если в ходе обсуждения рождается concrete решение, оно оформляется отдельным формальным запросом (!feat, !fix, #proto, #rule) с соответствующими требованиями (trace.log где нужно).

5. Теги состояния (@) — автотокены

Сигил «@» обозначает сведения о состоянии: версия, контрольная сумма, статус, среда, наличие файла-отчёта. Главная хитрость — вам не нужно помнить числа и длинные хеши. Вместо них вы пишете короткие слова-заместители (автотокены), а я подставляю реальные значения из состояния проекта.

Тег	Что описывает	Автотокены (вы пишете)	Что я подставлю
@v	Текущая версия / точка работы.	maxTask, milestone, или явное число	maxTask → последний номер задачи из журнала; milestone → замороженная v43.
@md5	Контрольная сумма (цифровой отпечаток файла архива).	latestZip, milestone	latestZip → отпечаток моего текущего архива; milestone → отпечаток замороженной копии.
@status	Статус версии: работает / сломана / неизвестно.	auto	auto → из журнала последний подтверждённый статус.
@env	Среда: разрядность, видеокарта, драйвер, ОС.	mine	mine → ваша закреплённая среда (x64, GTX 750, driver 560.94, Win11).
@trace	Прикреплён ли файл-отчёт trace.log.	auto	auto → «прикреплён», если файл есть в сообщении, иначе «отсутствует».

Таблица 4. Теги состояния и автотокены.

Главный выигрыш: вы не держите в голове номера версий и хеши. Пишете @v=maxTask — я сам нахожу последний номер задачи. Пишете @md5=latestZip — я сам считаю отпечаток и сверяю с вашим. Если отпечатки не совпали — моя песочница откатилась, и я сразу прошу переотправить архив, не начиная работу вслепую. Это и есть защита от потери работы.

6. meta.auto — файл автоподстановки

Чтобы автотокены работали, их правила где-то должны быть записаны. Для этого служит файл meta.auto — маленький конфигурационный файл рядом с журналом проекта. Я читаю его в начале каждого шага. Вот пример его содержимого:

```
# meta.auto — автоподстановка для sigil-протокола (ведёт ассистент)
```

```
[v]            default=maxTask          # max номер задачи из roadmap.md
[md5]          default=latestZip        # отпечаток download/demo_intro_latest.zip
[md5:milestone]  = c0cbce51ba1b11f90cbfc474c0b2d70a   # замороженная v43
[status]       default=auto             # из журнала: последний подтверждённый
[env:mine]       = x64,GTX750,driver560.94,Win11   # закреплено один раз
[trace]        default=auto             # прикреплён/отсутствует
[scope]        default=infer            # я вывожу из описания фичи
[avoid:default]  = opengl_loader.h, build_msvc.bat, build_mingw.bat   # НЕ трогать
[revert:default] = lastgood             # последняя рабочая версия
[prio:default]   = optional
[after:default]  = run.contains("First frame rendered") AND NOT
run.contains("[FATAL]") AND run.contains("[Exit] Main loop ended (code 0)")
[status:levels] = unverified → passed → confirmed   # A10: lastgood=only confirmed
[md5:canonical] = chat-printed    # A6: md5 in #ack=checkpoint; sandbox-md5
сверяется с последним
[btb:source]    = btb_stats.json   # BTB v2: delta-only, shadow_mode, Laplace
```

Теперь meta.auto — реальный файл в проекте (download/demo_intro/meta.auto). Рядом с ним живёт btb_stats.json — состояние BTB (см. раздел 7). Оба переживают откат песочницы вместе с roadmap.md — через zip и canonical-копию у вас.

Кто что пишет в этот файл:

Строка в meta.auto	Кто заполняет	Как часто меняется
[env:mine]	Вы говорите, я вписываю	Редко — только при смене видеокарты/драйвера
[md5:milestone]	Я, при заморозке новой контрольной точки	При каждой заморозке
[avoid:default]	Я, но substantive-правки — только после вашего подтверждения	Когда вскрывается новый «хрупкий файл»
[v]/[md5]/[status]/[trace] (default=...)	Никто числом — это правила, я разрешаю их на чтение	Никогда — следуют за состоянием проекта

Таблица 5. Кто и как обновляет meta.auto.

Ключевое отличие: meta.auto — это не обучаемая таблица, а декларативный конфиг (набор правил и закреплённых значений). Обучение на успехах и неудачах живёт в другом месте — в секции BTB журнала (см. следующий раздел).

7. BTB — предзагрузка по памяти

7.1. Как это работает

BTB — это моя внутренняя табличка-память. В ней записаны шаблоны (сигнатуры) ваших сообщений и списки файлов, которые мне скорее всего понадобятся. Например:

```
[btb] signature: "!fix main.cpp *" | "!feat *shader*"
     prefetch:  [read roadmap.md, grep main.cpp CrashHandler/_CRT/trace.log,
                 md5sum demo_intro_latest.zip, read opengl_loader.h(1..40)]
     confidence: 0.90   hits: 5   misses: 1

[btb] signature: "!diag *"
     prefetch:  [read upload/trace.log, diff vs trace_v45_GOOD_reference.log]
     confidence: 0.95   hits: 8   misses: 0
```

Когда приходит ваше сообщение, я сопоставляю его с сигнатурами. Если совпадение уверенное, я сразу запускаю чтение нужных файлов несколько штук одновременно (параллельно), пока ещё разбираю ваши точные слова. К моменту, как я понял задачу, файлы уже загружены. Без BTB мне пришлось бы читать их по одному — в три-четыре раза медленнее. Поток показан на диаграмме:



Диаграмма 2. Поток BTB-предзагрузки: сообщение → сигнатуры → параллельное чтение → разбор → действие. Промах = выкинуть прочитанное.

7.2. Почему это безопасно

Три жёстких правила защищают от неприятностей:

Только чтение. Заранее я только ЧИТАЮ файлы. Не пишу, не правлю, не собираю, не отправляю. Любые изменения — только ПОСЛЕ того, как я точно понял задачу. Неверная догадка ничего не портит — максимум чуть впустую потрачу на чтение.

Только лёгкие действия. Чтение, поиск, подсчёт отпечатка. Никаких тяжёлых операций.

Откат безопасен. Если догадка мимо — отбрасываю прочитанное (ничего не стоит) и разбираю запрос аккуратно по буквам.

7.3. Как обновляется BTB — файловый цикл (read-modify-write)

Важная оговорка: я не имею памяти между вашими сообщениями (stateless). Поэтому BTB НЕ может «сам в фоне» пересчитывать уверенность — на следующем промпте я бы забыл прошлый результат. Чтобы обучение было настоящим, BTB живёт в ФАЙЛЕ btb_stats.json, который я читаю в начале каждого хода и переписываю в конце (read-modify-write цикл):

```
1. Старт хода:   read meta.auto + btb_stats.json → match сигнатуры → prefetch (если conf≥0.6)
2. Работа:       выполняю запрос
3. Конец хода:   self-assess hit/miss → read-modify-write btb_stats.json → auto-prune conf<0.3
4. В ответе:     btb: signature X hit→H/M conf C   (прозрачность)
```

Уверенность (confidence) — число от 0 до 1 — растёт на попаданиях (hit) и падает на промахах (miss). Если после нескольких попыток уверенность опускается ниже 0.3 — запись автоматически удаляется при следующем write (подрезка). График ниже иллюстрирует динамику:

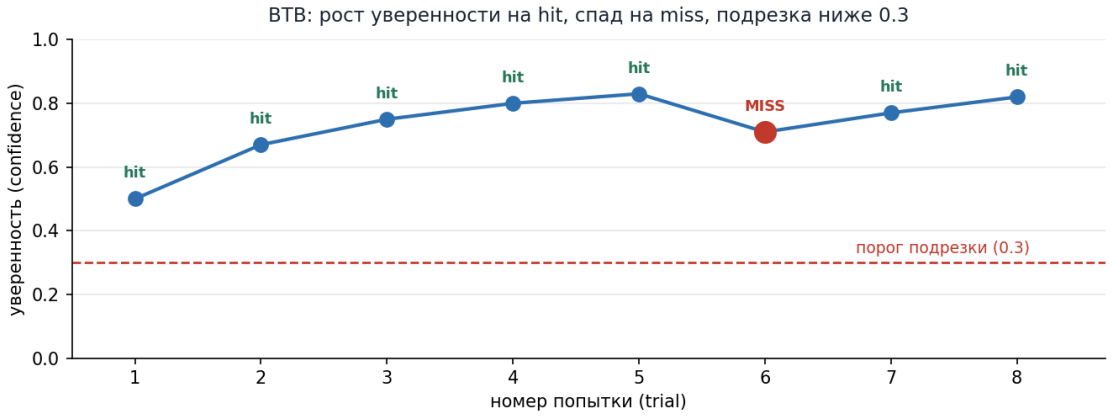


График 3. Уверенность BTB: рост на hit, спад на miss, ниже 0.3 — подрезка.

Событие	Что меняется в BTB	В остальных файлах
Успех (догадка верна, работа принята)	hits + 1, уверенность растёт	Журнал: новая задача, статус «работает»
Неудача (догадка мимо или работа сломана)	misses + 1, уверенность падает	Журнал: новая задача, статус «сломана», откат
Уверенность ниже 0.3 после нескольких попыток	Запись автоматически удаляется (подрезка)	—
Новый повторяющийся шаблон без записи	Я предлагаю новую запись через #proto	—

Таблица 6. Обучение BTB на успехах и неудачах.

7.5. #stats — метрики BTB

По команде #stats (или если прошло >7 дней с последнего отчёта — напоминаю) я читаю btb_stats.json и выдаю отчёт по обучению BTB:

```
BTB-evaluated turns: N   (из них матчинг сигнатуры)
  Hits: H   Misses: M   Hit-rate: H/(H+M)
Per-signature breakdown:
  !feat *shader* | !fix main.cpp *   H/M  conf C   [prefetch...]
  !diag *                           H/M  conf C   [prefetch...]
```

```
  ...
Top-3 prefetched файла (по частоте в prefetch-списках):
  1. roadmap.md — N сигнатур
  2. ...
  3. ...
Pruned entries (confidence<0.3): список из btb_stats.json.pruned[]
New signatures seeded this period: список
```

Важно: первые ~10-20 ходов после создания BTB числа — SEED-оценки из истории, не измеренные. Реальная запись началась с Task 49 (когда BTB принят). После разгона hit-rate и top-3 становятся устойчивыми метриками, а не оценками.

7.4. Честный предел

У меня нет памяти между вашими сообщениями: каждое сообщение будит меня заново. Поэтому я не могу начать читать файлы ДО того, как сообщение пришло — само сообщение и есть будильник. «Предзагрузка» происходит в первую секунду после сообщения: я запускаю несколько чтений сразу, а не по очереди. Выигрыш реальный, но скромный — это «параллельный разогрев», а не «предзагрузка до сигнала», как в процессоре.

7.6. BTB self-tuning: уточнение сигнатур через #proto

BTB — обучаемая система, и сигнатуры не фиксированы раз навсегда. Когда ход вскрывает over-fetch (прочитаны файлы, не понадобившиеся) или gap (нужен файл, не был в prefetch), я уточняю сигнатуру через #proto: split (разделить union-сигнатуру на узкие) или merge (объединить). Это цикл read-modify-write в действии: miss → diagnose → fix.

Пример (Task 52): сигнатура #proto|#ok|#stats имела union-prefetch [meta.auto, roadmap, btb_stats, docx-gen]. На #stats-ходе — over-fetch (meta.auto + docx-gen не понадобились). Split: #stats выделен в свою сигнатуру с tighter prefetch [btb_stats, roadmap]. Следующий #stats-ход — чистый hit (no gap, no over-fetch). Split валидирован за один ход.

Правило: split/merge предлагается через #proto, применяется после #ok. Каждое уточнение — BTB-запись с обновлённым prefetch и пересчитанной confidence. Так BTB «настраивается» на твой стиль не магией, а явными правками по сигналам с полей.

8. Полный синтаксис запроса !feat (и !remove)

Глаголы !feat и !remove — самые частые и самые богатые. !feat добавляет новую функцию, !remove хирургически удаляет одну. Структура у них одинаковая:

```
!feat | !remove  <описание фичи обычным языком>
  scope:   <ограничение, необязательно>
  after:   <короткий признак успеха/исчезновения, необязательно>
  revert:  lastgood          (необязательно — по умолчанию и так lastgood)
```

```
prio:    optional|required (необязательно — по умолчанию optional)
notes:   <свободный текст, необязательно>
```

Что вы пишете сами, а что подставляется из meta.auto:

Часть	Обязательно?	Кто пишет	Примечание
!feat/!remove <описание>	Да	Вы	Обычным языком, что и где сделать/удалить
scope:	Нет	Вы (только как ограничение)	По умолчанию я вывожу сам из описания
after:	Нет	Вы (только специфичный признак)	Успех по умолчанию = «First frame rendered + нет [FATAL] + [Exit] code 0»
revert:	Нет	По умолчанию lastgood	Можно не писать
prio:	Нет	По умолчанию optional	Пишите только если required
notes:	Нет	Вы, свободный текст	Без номеров задач — я сам маплю на журнал
@v/@md5/@status/ @env/@trace	Нет	Я, из meta.auto	Вы не пишете числа/хеши
avoid:/btb:	Нет	Я, из meta.auto/BTB	Вы не видите эти строки

Таблица 7. Обязательные и опциональные части запроса !feat/!remove.

На практике вы пишете минимум — описание и иногда признак успеха. Остальное подставляется автоматически. Для !remove признак успеха обычно про «исчезновение»: например, «nebula bg gone» — в trace.log больше нет milestone'а небулы.

8.1. after: как исполняемый предикат (DSL)

Важно: LLM плохо проверяют ОТСУТСТВИЕ в тексте — «нет milestone'а» легко просмотреть. Поэтому after: — это не описание, а ИСПОЛНЯЕМЫЙ предикат: когда вы присылаете trace.log после сборки, я буквально запускаю grep и сверяю счётчики. Никакого «прочитаю и оценю».

Примитив (v2.1)	Смысл	Пример
contains(STR)	fixed-string (-F) substring, весь trace.log	contains("First frame rendered")
run.contains(STR)	fixed-string в RUN-секции (CRLF-robust)	run.contains("[OK] nebula bg rendered")

Примитив (v2.1)	Смысл	Пример
build.contains(STR)	fixed-string в BUILD-секции	build.contains("error C")
NOT contains / NOT run.contains	отсутствие (fixed-string)	NOT run.contains("[FATAL]")
run.line(STR)	строка RUN точно=STR (ХРУПОК к CRLF — deprecated)	# не используй, бери run.contains
count(STR) == N	число СТРОК содержащих fixed-string STR	count("[OK]") == 3
AND / OR / NOT / ( )	булева; приоритет NOT > AND > OR	run.contains("X") AND NOT run.contains("Y")

Таблица 10. after-DSL v2.1: fixed-string (-F) БЕЗ regex-экранирования; scoped (run./build.); run.contains CRLF-robust; run.line deprecated.

Два execution-урока (Task 56 live-validation): L1 — под grep -F НЕ экранируй скобки (\[OK\] ищет литерал \[а не []; L2 — run.line (grep -Fx) хрупок к CRLF (Windows trace.log = \r) → run.contains. Точная строка '[Exit] Main loop ended (code 0)' безопасна от false-positive на 'code 0xC0000005' (там 0x...) не 0).

Семантика: все примитивы прошли → task status = working, BTB hit. Любой провалился → status = broken, revert: lastgood, репорт «какой предикат упал + фактический count». Это проверка WORK OUTCOME — отдельная ось от BTB hit/miss (точность prefetch'a).

Дефолтный контракт из meta.auto: contains("First frame rendered") AND NOT contains("[FATAL]") AND contains("[Exit]") AND contains("code 0").

9. Примеры использования

9.1. Простой вопрос

```
?почему вариадик-указатель ломает float ABI?
```

Ведущий «?» → обсуждение, файл-отчёт не требуется. Отвечаю сразу, без блока.

9.2. Новая функция (минимальная форма)

```
!feat nebula background in main fragment shader, replaces gradient+stars
  after: contains("[OK] nebula bg rendered")
```

Действие по коду, ведущего «?» нет → trace.log нужен. after задаёт специфичный признак успеха: в trace.log должна появиться строка [OK] nebula bg rendered.

9.3. Новая функция (детальная форма)

```
!feat nebula background (domain-warped FBM, 4 octaves; 5 light nodes) in main
fragment shader
```

```
  scope:  только шейдеры — не трогай загрузчик и build-скрипты
  after:  nebula bg rendered
  revert: lastgood
  prio:   optional
  notes:  палитра deep indigo + vibrant purple / electric cyan / neon orange / hot pink;
          без звёзд; мы уже делали небулу и откатили — сделай аккуратнее, не ломай
ACES
```

Та же задача, но с явными ограничениями и заметками. scope записан словами, без номеров строк — я сам маплю на конкретные файлы.

9.4. Удаление функции (новое в v0.2)

```
!remove nebula background from main fragment shader
  after: grep("nebula", trace.log) == 0 AND contains("First frame rendered")
  notes: верни gradient+stars как было до небулы; не трогай огонь и частицы
```

Хирургическое удаление одной фичи с сохранением всего остального. trace.log нужен (проверить, что после удаления программа не развалилась). Перед удалением — backup, так что всегда обратимо.

9.5. Диагностика сбоя

```
!diag чёрный экран после «Window shown — entering main loop»
```

Обязательно прикрепить trace.log. Я читаю его, ищу последний успех перед падением, сравниваю с эталонным хорошим trace.log — и называю причину за один шаг.

9.6. Откат к последней рабочей версии

```
!revert lastgood
```

lastgood — автотокен: я сам нахожу в журнале последнюю версию со статусом «работает». Вам не нужно помнить номер. trace.log не нужен — откат не правка кода.

9.7. Заморозка и новое правило

```
#milestone v45-nebula-working @md5=latestZip @status=working
#proto avoid:default += shaders_uniforms_table.h   (предложить)
```

#milestone — заморозить копию как эталон. #proto — предложить изменение протокола; substantive-правки требуют вашего подтверждения.

10. Правила взаимодействия

Поверх синтаксиса действуют три правила дисциплины обратной связи (записаны в журнале под номерами 12, 13, 14). Как они пропускают запрос — на диаграмме:



Диаграмма 4. Ворота правила 13: как запрос проходит или блокируется.

Правило	Когда срабатывает	Что происходит
12. trace.log — единый файл-отчёт	Всегда. Один файл = вывод компилятора + лог работы программы.	Вы присылаете его с каждым запросом по коду. В нём видно всё.
13. Нет trace.log → не выполнять запрос	Когда просите правку кода / диагноз, но не прикрепили trace.log (и сообщение не начинается с «?»).	Сразу вывожу «⚠️ Где trace.log? Атдай!» и отменяю запрос.
14. Ведущий «?» → правило 13 приостановлено	Когда сообщение начинается с «?».	Считаем обсуждением — trace.log не нужен. Если под «?» всё же правка кода — делаю, но помечаю, что проверка отложена.

Таблица 8. Три правила дисциплины обратной связи.

Эти правила — не формальность, а выжимка из реальной истории: мы потеряли много циклов на угадывание «что именно не работает», пока не ввели обязательный файл-отчёт и блок без него.

10.1. Kill-switch — ОТКЛОНЁН (Tasks 64 + reqs_killswitch.md)

В ходе работы над code booster (Tasks 63-64) была предложена концепция kill-switch: новый сигил «×» с глаголами ×disallow / ×allow, карантинная папка disallowed/, registry.json, скрипт disallow_artifact.py, и три новых правила (16, 17, 18) для safe rejection артефактов протокола. Подробная спецификация сохранена в /home/z/my-project/download/reqs_killswitch.md (status: REJECTED).

Решение пользователя: ОТКЛОНИТЬ kill-switch целиком. Принцип: не плодить инфраструктуру «про запас» (тот же принцип, что при отказе !macro в Task 53). Если в будущем потребуется отказаться от артефакта — это делается ad-hoc скриптом под конкретный случай, без общей инфраструктуры.

В протоколе остаётся 5 сигилов (? ! @ # &), без «×». Rules 16-18 НЕ активны. Папки disallowed/ НЕ существует. Все упоминания kill-switch в roadmap/журналах помечены как REJECTED, но сохранены для истории.

11. Словарь терминов

Все технические слова из этого документа и переписки — в одном месте, с расшифровкой.

Термин	По-русски	Короткое объяснение
protocol	правила общения	Набор соглашений о том, как мы пишем друг другу сообщения.
sigil	символ-метка	Значок в начале строки (?, !, @, #), обозначающий тип сообщения.
BTB (Branch Target Buffer)	буфер адресов переходов	Буфер в процессоре, который помнит, куда программа прыгала раньше. У нас — табличка-память шаблонов запросов.
prefetch	предзагрузка	Подготовить нужное заранее, до того как оно понадобится.
speculative execution	предварительное исполнение по догадке	Сделать что-то, предполагая результат; если угадал — сэкономил, если нет — откатываешь.
mispredict	промах догадки	Догадка оказалась неверной.
confidence	уверенность	Число от 0 до 1 — насколько сигнатура надёжно предсказывает нужные файлы.

Термин	По-русски	Короткое объяснение
hit / miss	попадание / промах	Догадка сработала / не сработала.
read-only	только чтение	Без записи и изменений — безопасно для предзагрузки.
write / commit	запись / зафиксировать	Изменения, которые остаются (в отличие от чтения).
rollback	откат	Вернуться к прошлой известной-хорошей версии.
rollback-safe	безопасный к откату	Действие, которое при неверной догадке можно безболезненно отменить.
parallel	параллельно	Несколько действий одновременно, а не по очереди.
latency	задержка	Время ожидания результата.
ad hoc	по ситуации	Без системы, импровизационно.
self-tuning	самонастраивающийся	Сам поправляет свои параметры на основе опыта.
prune	подрезка	Удаление слабых записей (с низкой уверенностью).
seed	затравка	Начальные записи, с которых стартует таблица.
intent	намерение	Чего хочет пользователь.
signature	сигнатура, узор	Шаблон сообщения, по которому ищут совпадение.
stateless	без памяти между сообщениями	Каждое сообщение будит меня заново, без воспоминаний о прошлом шаге.
milestone	контрольная точка, рубеж	Замороженная копия версии как эталон.
deliverable	готовая работа	То, что я отправляю вам (файл).
checksum / MD5	контрольная сумма, цифровой отпечаток	Короткая строка, однозначно идентифицирующая файл. По ней сверяют, совпадает ли моя копия с вашей.
diff	сравнение	Сопоставление двух файлов (например, хорошего и плохого лога).
warm-up	разогрев	Подготовка контекста перед работой.

Термин	По-русски	Короткое объяснение
sandbox	песочница	Моё рабочее окружение, где исполняются инструменты. Может откатываться.
trace.log	файл-отчёт	Единый файл: вывод компилятора + лог работы программы. Главный канал диагностики.
roadmap	журнал памяти проекта	Файл с задачами, статусами и правилами. Переживает перезапуск песочницы.
autotoken	автотокен	Короткое слово-заместитель (maxTask, latestZip ...), которое я разворачиваю в реальное значение.
code booster	ускоритель кода	ngram-статистика GLSL/Cg стиля (booster_final.json v1.1, 3674 контекста). Вставляется в промпт при !*shader* запросах — advisory hints для генерации.
llm_layers	слои для LLM	Секция booster_final.json v1.1: static_block + rules_compact + patterns_full + nesting_full + arity_full + meta_quality. Загружается в prompt как L1 layer (~2500 токенов).
compat_layer	слой совместимости	Секция booster v1.1: mapping Cg → GLSL 3.3 (tex2D→texture, float4→vec4, mul(a,b)→((a)*(b)) и т.д., 14 entries). Применяется ДО интерпретации hints.
query_api	API запросов	Секция booster v1.1: dynamic lookup ngram.contexts через scripts/query_booster.py. Возвращает top-k кандидатов на следующий токен. Не для prompt injection.
validation_pipeline	конвейер валидации	5 слоёв: L1 style (booster static) + L2 dynamic (query_api) + L3 syntax (glslangValidator) + L4 runtime (after-predicate) + L5 self_test (mental check).

Термин	По-русски	Короткое объяснение
glslangValidator	валидатор GLSL	Официальный валидатор Khronos. У нас v15.2.0, запускается через scripts/validate_glsl.py перед доставкой шейдера. errors=blocking, warnings=advisory.
aesthetic_notes	эстетические заметки	YAML-блок в ответе на ! *shader*: palette, ratio, harmony, focal_point, motion, density, contrast, reference_target, self_critique. Mandatory для v0.5+ pipeline.
DRAMATIC_DARK	палитра DRAMATIC_DARK	Sample палитра из aesthetics.md §3.4: deep_purple_core + cyan_mid + magenta_rim + star_white_pure + void_black.
archetype	архетип	Эстетический тип изображения: static surreal / dynamic dramatic / atmospheric horror. Из comparative analysis aesthetics.md §9.6.
principled verdict	принципиальный вердикт	Выбор между изображениями на основе principles из aesthetics.md, не на основе taste. Всегда с honest caveat: principles ≠ absolute truth.
kill-switch	выключатель артефакта	ОТКЛОНЁН (reqs_killswitch.md REJECTED). Предлагался как сигил × с глаголами ×disallow/×allow для отказа от артефактов протокола. Не реализован.

Таблица 9. Словарь англицизмов и терминов (v1.5: +12 новых терминов для booster/aesthetics).

12. Краткая шпаргалка

Всё самое главное на одной странице — для быстрого взгляда.

Сигилы

Сигил	Когда ставить	Одной строкой
?	Вопрос, обсуждение	trace.log не нужен
!	Действие (правка, фича, удаление, диагноз, откат)	trace.log нужен, иначе блок
@	Факт о состоянии	Я сверяю с песочницей
#	Правило, подтверждение, заморозка	Записываю в журнал
&	Предложение/обсуждение методов (Task 62)	trace.log не нужен, BTB delta=[]

Глаголы для ! (обязательны внутри !)

```
!fix  !feat  !remove  !revert  !diag  !snapshot  !verify  !clean
```

trace.log нужен для: !fix !feat !remove !diag. Не нужен для: !revert !snapshot !verify !clean.

Глаголы для & (Task 62, v1.5)

```
&suggest  &review  &plan  (trace.log НЕ нужен, BTB delta=[])
```

Мета-команды

```
#rule  #milestone  #proto  #stats  #ok  (#stats — отчёт метрик BTB)
```

Автотокены (вместо чисел и хешей)

```
@v=lastBuild      → последняя версия КОДА (не maxTask — задачи 49+ протокольные)
@task=maxTask     → последний номер задачи (счётчик, отдельный от версии)
@md5=latestZip    → отпечаток текущего архива
@md5=milestone    → отпечаток замороженной копии
@status=auto      → из журнала
@env=mine         → ваша закреплённая среда
@trace=auto       → прикреплён/отсутствует
revert: lastgood  → последняя рабочая версия
```

Три правила

12. trace.log — единый файл-отчёт, присылать всегда.

13. Нет trace.log (и нет «?») → «Где trace.log? Атдай!» + отмена.

14. Ведущий «?» → правило 13 приостановлено.

Минимальный запрос на новую функцию

```
!feat <что сделать, обычным языком>
  after: <короткий признак успеха>
```

Остальное подставится само. Не забудьте прикрепить trace.log.

13. v2 reforms summary (Tasks 49-56)

Краткая сводка эволюции протокола с v1.3 → v1.4. Подробности — в roadmap.md (Tasks 49-56).

Реформа	Task	Что изменилось
BTB v2: delta-only prefetch	55 (A5)	base_set [meta.auto, btb_stats, roadmap] убран из BTB-entries; BTB трекает только signature-specific delta. ?* и #stats retired (base-only).
BTB v2: shadow_mode	55 (A2)	evaluate на каждом матчатом ходе, даже ниже порога 0.6 → нет мёртвой зоны 0.3-0.6.
BTB v2: Laplace confidence	55 (A2)	(seed_h + meas_h + 1)/(seed_n + meas_n + 2). seed vs measured разделены.
BTB = round-trip batching	55 (A4)	reframe: не parallel-with-parsing (LLM читает сообщение целиком, потом tool-calls), а batching. Выигрыш скромный, не 3-4×.
md5:canonical (chat-printed)	55 (A6)	md5 в #ack-блоке = checkpoint; sandbox-md5 сверяется с последним напечатанным. Chat не откатывается. 'Нет памяти' → 'нет cross-session памяти; within-chat history видна'.
after-DSL v2.1: fixed-string	55/56 (A7)	grep -F (fixed-string) БЕЗ regex-экранирования; scoped (run.contains/build.contains); run.line CRLF-fragile → deprecated; precedence NOT>AND>OR.

Реформа	Task	Что изменилось
status: unverified→passed→confirmed	55 (A10)	lastgood = only confirmed (user @status=confirmed или #milestone), НЕ after-passed. Защита от 'log-passed но чёрный экран'.
BTB v2 live-validated	56	!feat probe: md5 MATCH, delta-only, shadow-eval recall 1.0/precision 0.75 HIT, Laplace 0.63→0.667. 2 execution-бага пойманы (L1, L2) → v2.1.

Таблица 11. v2 reforms (Tasks 49-56) — что изменилось с v1.3.

v1.5: задачи 57-66 продолжают эволюцию протокола. См. таблицу ниже.

13.1. v1.5 расширение (Tasks 57-66)

Сводка изменений с v1.4 → v1.5. Подробности — в roadmap.md (Tasks 57-66).

Task	Что сделано	Влияние на протокол
57	docx v1.4: cover v1.4; meta.auto example → v2.1 (run.contains + status:levels + md5:canonical + btb:source); Table 10 → after-DSL v2.1 (fixed-string -F, scoped run.contains CRLF-robust, run.line deprecated, precedence NOT>AND>OR) + L1/L2 lessons; cheat sheet per-verb trace.log + @v=lastBuild/@task=maxTask; section 13 'v2 reforms summary' (Таблица 11). 20 стр, 12 табл.	Документация v1.4 formalised everything from Tasks 49-56.
58	#milestone v47 (user): v47 → confirmed (A10). demo_intro_v47_WORKING.zip заморожен (MD5 b0b6413...). lastgood → v47.	Второй замороженный milestone (после v43).

Task	Что сделано	Влияние на протокол
59	GLSL validator pipeline (code booster, user #ok): glslangValidator 15.2.0 (apt-get download + dpkg-deb -x, без root). scripts/validate_glsl.py — 4 шейдера, glslang OpenGL mode. Протокол: перед ! *shader* delivery run validator; errors=blocking, warnings=advisory. meta.auto += [glsl:validator]. noise3 rename → !fix v48.	Pre-delivery syntax validation pipeline formalised. Catches built-in collisions (как noise3 в v47).
60	!fix rename noise3→fbmNoise3 (user, trace.log v47, rule 13 ✓): shaders.h — 2 spots (def + fbm3 call). GLSL validator ДО доставки: 4/4 PASS (noise3 collision resolved). v47→v48. after-PASSED ✅ (4/4 predicates).	First full code-booster loop closed (validator catches pre-delivery → fix → validator confirms → after confirms runtime).
61	#milestone v48 (user): v48 → confirmed (A10). demo_intro_v48_WORKING.zip заморожен (MD5 0906644...). lastgood → v48.	Третий замороженный milestone. BTB GAP identified: ! fix *shader* signature не покрыта.
62	#proto: НОВЫЙ СИГИЛ `&` + глагол `&suggest` (user #ok). Назначение: обсуждение методов оптимизации работы, предложения по улучшению. trace.log НЕ нужен (как ?). Глаголы: &suggest &review &plan. BTB: delta=[] (base-only).	Протокол расширен с 4 до 5 сигилов (? ! @ # &). 3 новых глагола для &.
63	code booster v1.0: booster_final.json разобран (74 КБ). 3 674 контекста ngram, 7 правил (R-01..R-06 hard + S-01 soft), 40 arity-функций, 229 vocab, 40 patterns, 9 test_cases. llm_prompt_block (31 строка) — готовый текст для вставки в промпт. Качество: top1=54%, top3=77%, top5=85%.	Code booster v1.0 добавлен в протокол как advisory layer для !*shader* задач.

Task	Что сделано	Влияние на протокол
64	#ok booster-still-on (user): 6 расширений booster_final.json реализованы БЕЗ kill-switch. booster v1.0 → v1.1: compat_layer + llm_layers + query_api + llm_self_test + not_for_llm_prompt + validation_pipeline. scripts/apply_booster_v2.py (patcher) + scripts/query_booster.py (dynamic ngram lookup). Self-test: 4/5 complete-case HIT. Kill-switch ОТКЛОНЁН (reqs_killswitch.md REJECTED).	Code booster v1.1 full integration. 6 new JSON sections + 2 Python scripts. Kill-switch rejected (no × sigil, no Rules 16-18).
65	!fix *shader* nebula upper-cyan (user, trace.log v48, rule 13 ✓): NebulaBackground diagnosis — upper half (uv.y>0.5) имела только q1 orange + q2 magenta, NO cyan node. Fix: (1) col-mix smoothstep(0.4, 0.8, n*n) → smoothstep(0.3, 0.7, n), (2) added q5 cyan node at (0.5, 0.75). v48→v49. Booster pipeline v1.1 first end-to-end run: L1 loaded + L2 2 calls HIT (hit_rate 2/2) + L3 mental skip + L4 validator 4/4 PASS + L5 after-predicate PENDING. after-PASSED 5/5 ✓ (trace.log v49).	First real !*shader* task on booster v1.1. Full pipeline cycle closed. BTB !fix *shader* first measured entry, confidence 0.75.
66	#milestone v49 (user): v49 → confirmed (A10). demo_intro_v49_WORKING.zip заморожен (MD5 6539b119..., 171 КБ). lastgood → v49.	Четвёртый замороженный milestone. Booster v1.1 first cycle archived in frozen zip.

Таблица 11b. v1.5 расширение (Tasks 57-66) — что изменилось с v1.4.

Итог v1.4 → v1.5: +1 сигил (&), +3 глагола (&suggest/&review/&plan), +code booster v1.1 (6 JSON секций + 2 Python scripts + first end-to-end cycle closed), +GLSL validator pipeline (glslangValidator 15.2.0), +3 новых milestone (v47/v48/v49), +aesthetics.md/docx (v0.4 cleaned, 3-image comparative analysis), kill-switch ОТКЛОНЁН. Protocol остаётся lean: 5 сигилов, 8+3 глаголов, без макросов и без автоматизированной инфраструктуры отказа.

14. Code booster v1.1 — GLSL style statistics

Code booster — это ngram-статистика GLSL/Cg стиля, вычисленная по реальному репозиторию (Demoscene project, 103 уникальных файла, 85 928 токенов). На основе booster'a я делаю advisory hints при генерации шейдерного кода. Booster НЕ заменяет glslangValidator (blocking) — это мягкий слой для style alignment.

14.1. Booster v1.0 → v1.1 — 6 расширений (Task 64)

Booster v1.0 содержал только llm_prompt_block (31 строка) — готовый текст для вставки в промпт. Booster v1.1 (Task 64, user #ok booster-still-on) добавил 6 новых секций, чтобы использовать файл целиком, а не только одну секцию:

Секция	Назначение	Применение
compat_layer	Mapping Cg → GLSL 3.3 (14 entries: tex2D→texture, float4→vec4, mul(a,b)→((a)*(b)), :COLOR→out vec4 fragColor и т.д.)	Применяется ДО интерпретации hints. Cg-лексемы в patterns/nesting auto-translate.
llm_layers	Consolidated LLM-facing data: static_block (31) + rules_compact (7) + patterns_full (40) + nesting_full (28) + arity_full (40) + meta_quality	L1 layer — загружается в prompt как ~2500 токенов. Static advisory hints.
query_api	Dynamic lookup spec: interface, triggers T1-T4, hard_cap 15 calls/task, fallback path	L2 layer — динамические queries через scripts/query_booster.py. NOT for prompt injection.
llm_self_test	9 compact test cases (T-C1..T-C5 complete + T-K6..T-K9 check) для mental self-check	L5 layer — post-generation mental check на known patterns.
not_for_llm_prompt	Exclusion notes: ngram.contexts (~150KB, too large для prompt), algorithm, normalization, vocab_full	Документация: что НЕ вставлять в prompt, даже если хочется.
validation_pipeline	5-layer diagram: L1 style + L2 dynamic + L3 syntax (validator) + L4 runtime (after) + L5 self_test	Карта pipeline. Precedence: layer N+1 overrides layer N.

Таблица 12. Code booster v1.1 — 6 новых секций (schema_version 1.0 → 1.1).

Canonical path: download/demo_intro/booster_final.json (948 КБ, working copy). Original upload/booster_final.json (328 КБ, v1.0) сохранён как read-only source. Patcher: scripts/apply_booster_v2.py (idempotent, воспроизводит v1.1 из v1.0).

14.2. 5-layer validation pipeline

Booster v1.1 встраивается в существующий pipeline валидации. 5 слоёв, каждый со своей ролью и severity:

Слой	Имя	Tool	Когда	Severity	Cost
L1	style	llm_layers.static_block + compat_layer	pre-generation (loaded once per *shader* task)	advisory	~2500 tok
L2	dynamic	query_api → scripts/query_booster.py	mid-generation, per-statement at triggers T1-T4	advisory	0-750 tok (50 per query × 0-15)
L3	syntax	glslangValidator via scripts/validate_glsl.py	pre-delivery	blocking	0
L4	runtime	after-predicate DSL v2.1 on trace.log	post-execution (user sends trace.log)	verification	0
L5	self_test	llm_self_test (9 cases, mental check)	post-generation, before delivery	advisory	0

Таблица 13. 5-layer validation pipeline (booster v1.1 integration).

Precedence: L3 (blocking) overrides L1+L2 (advisory). L4 (runtime verification) overrides всё. L5 — sanity check, не blocking.

14.3. query_booster.py — CLI interface

Dynamic ngram lookup реализован Python-скриптом. Tokenize partial GLSL → lookup ngram.contexts (3674 entries) с Katz backoff → apply hard rules R-01..R-06 + soft S-01 → return top-k candidates с probabilities.

```
python3 scripts/query_booster.py --code '<partial GLSL>' \
  [--stage fragment|vertex|compute] [--top 5] [--self-test] [--show-context]
```

Triggers T1-T4 (когда запускать query в ходе генерации):

Trigger	Когда срабатывает	Пример
T1	Choice point с ≥3 plausible candidates	После `clamp(` — может быть VAR/expr/pow/sin. Query даёт top-3: expression 77%, pow 8%, sin 7%

Trigger	Когда срабатывает	Пример
T2	Unfamiliar context (function/pattern не в memory)	New SDF primitive — не помню стиль проекта. Query подсказывает вероятное continuation.
T3	Deep nesting ≥ 3 levels of parens	mix(clamp(dot(N,L),0,1), ...) — 3+ уровня. ngram-4 даёт real signals.
T4	После !fix root-cause = style anomaly	noise3 collision в v47. После fix'a query на 2-3 ключевых местах, чтобы убедиться, что других аномалий нет.

Таблица 14. query_booster.py triggers — когда запускать L2 dynamic lookup.

Hard cap: 15 calls per !*shader* task. Если превышен — pause, перепроверить подход (возможно, нужен &plan с пользователем, а не гадать через query). Fallback: если script недоступен — продолжать с L1 static + L5 self-test только.

14.4. GLSL validator pipeline (Task 59)

Pre-delivery blocking layer. glslangValidator 15.2.0 (Khronos official). Установка: apt-get download + dpkg-deb -x (без root). Запуск: scripts/validate_glsl.py — извлекает 4 shader source (kVertexShaderSrc, kFragmentShaderSrc, kFireVertexShaderSrc, kFireFragmentShaderSrc) из shaders.h, передаёт each в glslangValidator в OpenGL mode.

```
# Протокол: перед каждым !*shader* delivery
python3 scripts/validate_glsl.py
# output:
#   ✓ kVertexShaderSrc          [vert]  PASS
#   ✓ kFragmentShaderSrc        [frag]  PASS
#   ✓ kFireVertexShaderSrc      [vert]  PASS
#   ✓ kFireFragmentShaderSrc    [frag]  PASS
#   === ALL PASS ===

# errors = BLOCKING (delivery отменяется)
# warnings = advisory (delivery продолжается, отмечается в ответе)
```

Live demo: поймал noise3 collision в v47 (kFragmentShaderSrc FAIL — noise3 это GLSL built-in). v48 rename noise3→fbmNoise3 → 4/4 PASS. Первый full code-booster loop closed (validator catches pre-delivery → fix → validator confirms → after confirms runtime).

14.5. First end-to-end cycle (Task 65 — v49 upper-cyan-fix)

Первая реальная !*shader* задача на booster v1.1. v49 upper-cyan-fix — diagnosed NebulaBackground upper half (uv.y>0.5) имела только q1 orange + q2 magenta, NO cyan node. Fix: col-mix smoothstep change + new q5 cyan node at (0.5, 0.75).

Слой	Result	Detail
L1 static	✅ LOADED	llm_layers (31 static_block + 7 rules + 40 patterns + 28 nesting + 40 arity + meta_quality) + compat_layer. ~2500 tok в working context
L2 dynamic	✅ 2 calls HIT	T1 trigger (choice point for fix approach). Query #1: confirmed <NUM> (0.34) typical after `*` → my `* 0.5` matches existing pattern HIT. Query #2: verified q5 line → top-1 <NUM> (0.42) → my `0.5` HIT. hit_rate 2/2 = 1.000
L3 self-test	⏸️ mental skip	9 cases T-C1..T-C5 syntax-level, не relevant для color placement fix. T-K6..T-K9 check-action, не запускаются. Skip обоснован
L4 syntax	✅ BLOCKING PASS	glslangValidator 15.2.0 — 4/4 PASS (kFragmentShaderSrc with q5 added OK)
L5 runtime	✅ 5/5 PASS	after-predicate run.contains('[OK] nebula bg rendered') AND NOT run.contains('[FATAL]') — trace.log v49 contains both markers (×2 for 2 RUN START sections), no FATAL. Default predicate: FirstFrame ✓ + Exit code 0 ✓ + no FATAL ✓

Таблица 15. First end-to-end booster v1.1 cycle (Task 65, v49 upper-cyan-fix).

Booster pipeline v1.1 — FULL CYCLE CLOSED. BTB !fix *shader* first measured entry: delta = [shaders.h:kFragmentShaderSrc, booster_final.json, scripts/validate_glsl.py, md5:latestZip], seed = 1h/0m, measured = 1h/0m, confidence = 0.75 (Laplace), last_result = hit.

14.6. BTB entry для !fix *shader* (first measured)

BTB v2 entry для сигнатуры !fix *shader* (создана в Task 65, first measured). 5 entries total в btb_stats.json v2 после Task 65.

```
{
  "signature": "!fix *shader*",
  "delta": ["shaders.h:kFragmentShaderSrc", "booster_final.json",
            "scripts/validate_glsl.py", "md5:latestZip"],
  "seed":     {"hits": 1, "misses": 0},
  "measured": {"hits": 1, "misses": 0},
  "confidence": 0.75,   # Laplace: (1+1+1)/(1+1+2) = 3/4
```

```
  "last_result": "hit",
  "note": "First !fix *shader* measured entry (Task 65). Booster L1+L2+L4+L5 closed."
}
```

GAP (Task 61 note): existing signature «!feat *shader* | !fix main.cpp *» не покрывает !fix *shader*. Suggested #proto (future): extend to «!feat *shader* | !fix *shader*» (same delta — shaders.h + booster + validator). Pending refinement via Task 52 signature split/merge mechanism.

15. v1.5 changelog — что нового (Tasks 57-66)

Краткий список изменений v1.4 → v1.5:

1. Сигил & (Task 62). Новый 5-й сигил — & — для предложений и обсуждения методов. 3 глагола: &suggest, &review, &plan. trace.log не нужен. BTB delta=[].

2. Code booster v1.0 разобран (Task 63). booster_final.json (74 КБ) — ngram-статистика GLSL/Cg стиля, 3 674 контекста, 7 правил. top1=54%, top3=77%, top5=85%.

3. Code booster v1.1 — 6 расширений (Task 64). compat_layer + llm_layers + query_api + llm_self_test + not_for_llm_prompt + validation_pipeline. Schema 1.0 → 1.1. Scripts: apply_booster_v2.py (patcher) + query_booster.py (dynamic lookup).

4. Kill-switch ОТКЛОНЁН. Сигил × с глаголами ×disallow/×allow, Rules 16-18, disallowed/ папка — предложены в reqs_killswitch.md, REJECTED пользователем. Принцип: не плодить инфраструктуру «про запас» (как с !macro в Task 53).

5. First booster v1.1 cycle closed (Task 65). !fix *shader* nebula upper-cyan на v49: L1 loaded + L2 2/2 HIT + L3 skip + L4 4/4 PASS + L5 5/5 PASS. BTB !fix *shader* first measured entry, conf=0.75.

6. v49 milestone (Task 66). demo_intro_v49_WORKING.zip заморожен (MD5 6539b119..., 171 КБ). Четвёртый milestone (v43/v47/v48/v49). lastgood → v49.

7. Aesthetics doc создан. aesthetics.md v0.4 (316 строк) — pure principles без project-specific refs. 3-image comparative analysis (warehouse surreal / mage combat / zombie horse graveyard). docx generated (508 КБ, 10 pages, 3 embedded images).

8. GLSL validator pipeline formalised (Task 59). glslangValidator 15.2.0 через scripts/validate_glsl.py. Pre-delivery blocking layer. meta.auto += [glsl:validator]. Live: caught noise3 collision в v47 → v48 fix.

9. meta.auto расширена. + [booster:v2], [booster:canonical], [booster:query], [booster:patcher], [booster:first-measured], [v:49], [md5:v49], [md5:v48] (milestone frozen).

10. Документация расширена. §3 Сигилы: 4 → 5. §4.2: новый подраздел для & глаголов. §10.1: note про kill-switch REJECTED. §11: +12 новых терминов. §13.1: v1.5 extension (Tasks 57-66). §14: НОВЫЙ раздел про code booster v1.1 (6 подразделов). §15: этот changelog.

Протокол остаётся lean: 5 сигилов (? ! @ # &), 8+3 глаголов, без макросов, без автоматизированной инфраструктуры отказа. Принцип «не расширять про запас» соблюдён.